SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Mannepalli Satwik	**Reg. No.:**	192424339
Date:	16.10.2026	**Duration:**	60 minutes

Code of Conduct

I Mannepalli Satwik (192424339)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Satwik

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25

Industry Context: The explosive growth of low-Earth orbit (LEO) satellite constellations has overwhelmed legacy ground station tracking networks. Ground stations must lock onto passing satellites, upload system commands, and download massive payloads of earth observation data during tiny, minutes-long orbital windows. Mechanical component wear, atmospheric interference, and scheduling conflicts require an intelligent tracking network that maximizes data downloads and schedules access automatically.

Problem Statement: Design and develop a distributed Satellite Ground Station Telemetry & Scheduling Network that optimizes contact windows across international satellite tracking infrastructure. The system must ingest high-velocity data points, including orbital mechanic trajectories, satellite health logs, ground antenna alignment precision, and local atmospheric distortion indexes. It must schedule satellite-to-station pairings dynamically and adjust tracking hardware on the fly to maximize data download throughput.

DISTRIBUTED SATELLITE GROUND STATION TELEMETRY & SCHEDULING NETWORK

Agile Sprint Planning and User Story Workshop

1. Introduction

1.1 Project Overview

The Distributed Satellite Ground Station Telemetry and Scheduling Network is a software system designed to coordinate satellite communication activities across multiple geographically distributed ground stations. Ground stations communicate with satellites to receive telemetry information, send commands where authorized, and collect mission-related data.

When several satellites and ground stations operate simultaneously, scheduling communication windows becomes a complex task. Each satellite is visible to a ground station only during a particular time window. If two satellites require the same ground station at overlapping times, a scheduling conflict may occur. Similarly, a ground station may experience network failures, equipment problems, or unexpected data delays.

The proposed system provides a centralized platform to manage ground-station availability, satellite contact windows, telemetry ingestion, station health, alerts, and scheduling decisions. It helps operators make informed decisions by presenting relevant information through a dashboard.

The project follows an Agile Scrum methodology. The system is divided into manageable features, represented as epics and user stories. These stories are prioritized in a product backlog and selected for implementation during sprint planning.

1.2 Problem Statement

Traditional satellite ground-station coordination may involve separate scheduling tools, manual status checks, and communication between different operational teams. When information is distributed across systems, operators may face difficulties identifying overlapping contact windows, detecting station failures, and responding to schedule changes.

The main problems are:

- Difficulty coordinating multiple ground stations.
- Conflicts between satellite contact windows.
- Delayed identification of station or communication-link failures.
- Inconsistent telemetry data and missing information.
- Limited visibility into station availability and operational status.
- Delayed communication of schedule changes and system alerts.
- Challenges in controlling access to sensitive operational information.

Therefore, an integrated system is required to improve scheduling, monitoring, data validation, and operational coordination.

1.3 Proposed Solution

The proposed solution is a web-based platform that integrates scheduling, telemetry monitoring, ground-station health information, notifications, and access control.

The system will:

1. Register and manage ground stations.
2. Receive and validate telemetry information.
3. Maintain satellite contact-window information.
4. identify scheduling conflicts.
5. Generate suitable scheduling recommendations.
6. Monitor station health and communication-link status.
7. Notify authorized users about important events.
8. Provide dashboards and reports for operational decision-making.
9. Protect system information through authentication and authorization.

The initial implementation will use simulated satellite data and contact windows where real satellite infrastructure is unavailable. This makes the project suitable for an academic prototype.

2. Project Vision and Objectives

2.1 Project Vision

To develop a reliable, secure, and scalable platform that improves the coordination of distributed satellite ground stations through intelligent scheduling, telemetry monitoring, and timely operational alerts.

2.2 Project Objectives

The major objectives are:

1. Centralized coordination: Maintain ground-station details, availability, and operational status in one platform.
2. Efficient scheduling: Identify overlapping contact windows and recommend conflict-free schedules.
3. Telemetry management: Collect, validate, and store telemetry data in a structured format.
4. Health monitoring: Track station and communication-link conditions.
5. Early alerting: Inform operators when abnormal readings, delays, or failures are detected.
6. Secure access: Ensure that users can access only the functions permitted by their roles.
7. Operational visibility: Present scheduling information, station status, and alerts through dashboards.
8. Agile development: Deliver the system incrementally through short development sprints.
9. Testability: Define measurable acceptance criteria and verify the implementation through testing.
10. Future scalability: Design the system so that additional stations and monitoring capabilities can be incorporated later.

2.3 Expected Outcomes

At the end of the project, the expected outcomes are:

- A working prototype of the ground-station management dashboard.
- A structured database for station, telemetry, and scheduling information.
- A scheduling module that detects overlapping contact windows.
- A telemetry validation and health-monitoring module.
- An alert and notification mechanism.
- A role-based access-control mechanism.
- A tested and documented application developed through Agile sprints.

The academic prototype demonstrates the intended workflow. It does not replace certified satellite-control infrastructure or guarantee real-world spacecraft communication.

3. Stakeholders and Their Responsibilities

A stakeholder is a person or group that influences the project or is affected by its results.

Stakeholder	Responsibility
Ground-station operator	Reviews station availability and executes approved schedules
Mission planner	Defines satellite contact requirements and scheduling priorities
System administrator	Manages user accounts, roles, and system configuration
Network engineer	Monitors communication links and network health
Telemetry analyst	Reviews telemetry readings and identifies unusual values
Software development team	Designs, develops, and tests the application
Product owner	Defines priorities and approves completed features
Scrum master	Facilitates Scrum activities and removes process obstacles
Security administrator	Reviews access permissions and security events

3.1 Stakeholder Requirements

Each stakeholder has different expectations.

- Operators require clear schedules and immediate visibility of conflicts.
- Mission planners require reliable contact-window information.
- Network engineers require station-health indicators and alerts.
- Telemetry analysts require validated, searchable telemetry records.
- Administrators require secure account and permission management.
- Project evaluators require demonstrable functionality, testing evidence, and documentation.

Understanding these requirements helps the development team create useful features and prioritize them appropriately.

4. Project Scope and Agile Development Approach

4.1 Scope of the Project

The project scope defines the features that will be implemented in the proposed system.

Features included in the project

1. Ground-station registration and status management.
2. Satellite contact-window entry and management.
3. Telemetry data ingestion and validation.
4. Scheduling conflict detection.
5. Schedule recommendation and approval.
6. Station-health and communication-link monitoring.
7. Alerts and notifications.
8. Dashboard and report generation.
9. User authentication and role-based access control.
10. Testing and documentation.

Features excluded from the initial version

- Direct control of real satellites.
- Transmission of actual spacecraft commands.
- Integration with every real-world ground-station provider.
- Fully autonomous mission planning.
- Certified aerospace safety systems.
- Advanced artificial intelligence-based satellite trajectory optimization.

These features may be considered in future versions if the necessary infrastructure, permissions, and resources are available.

4.2 Assumptions

The project is developed using the following assumptions:

- Ground-station information will be available in a structured format.
- Satellite contact windows can be entered manually or generated using simulated data.
- Telemetry records will contain timestamps and measurement values.
- The initial version will be demonstrated using sample data.
- Users will have suitable devices and network access to use the web application.
- Scheduling recommendations will be reviewed by authorized operators before approval.
- The project team will have access to a development environment and a database.

4.3 Agile Scrum Methodology

Agile is a software development approach that emphasizes iterative development, continuous feedback, teamwork, and incremental delivery.

Scrum is an Agile framework in which work is divided into short periods called sprints. Each sprint produces a potentially usable increment of the software.

The main Scrum elements are:

- Product Backlog: A prioritized list of features and requirements.
- Sprint Planning: The meeting where the team selects work for the upcoming sprint.
- Sprint Backlog: The user stories and tasks selected for the sprint.
- Daily Scrum: A short meeting to review progress and identify obstacles.
- Sprint Review: A demonstration of completed work to stakeholders.
- Sprint Retrospective: A meeting to identify improvements for the next sprint.
- Increment: The tested functionality completed during a sprint.

4.4 Why Agile Is Suitable for This Project

Agile is suitable because the project contains several independent but connected modules. The team can develop and demonstrate each module incrementally rather than waiting until the entire system is finished.

For example, the first sprint can deliver station registration, the second can deliver scheduling, and a later sprint can introduce telemetry monitoring.

Agile also supports feedback. If operators find the scheduling dashboard difficult to understand, the team can improve it during a later sprint.

5. System Architecture and Process Diagrams

5.1 High-Level System Architecture

The architecture describes how the major components of the application communicate with each other.

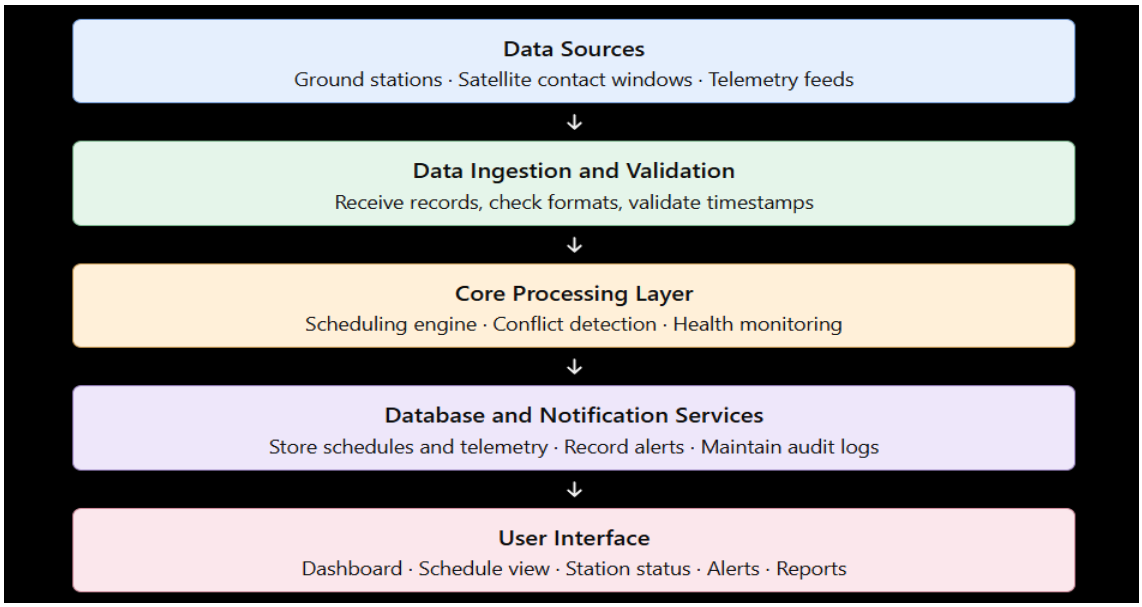


Figure 1: High-Level Architecture of the Proposed System

Data Sources

Ground stations · Satellite contact windows · Telemetry feeds

Data Ingestion and Validation

Receive records, check formats, validate timestamps

Core Processing Layer

Scheduling engine · Conflict detection · Health monitoring

Database and Notification Services

Store schedules and telemetry · Record alerts · Maintain audit logs

User Interface

Dashboard · Schedule view · Station status · Alerts · Reports

Explanation

1. Data Sources: Supply station information, satellite contact windows, and telemetry records.
2. Data Ingestion and Validation: Checks incoming data for missing fields, invalid values, and incorrect timestamps.
3. Core Processing Layer: Identifies scheduling conflicts and evaluates station-health information.
4. Database and Notification Services: Store validated information and record relevant events.
5. User Interface: Displays schedules, operational status, and alerts to authorized users.

The architecture separates the user interface from the processing logic and data storage. This makes the system easier to maintain and extend.

5.2 Scheduling Workflow

The scheduling module identifies possible contact windows and checks whether the proposed assignments conflict.

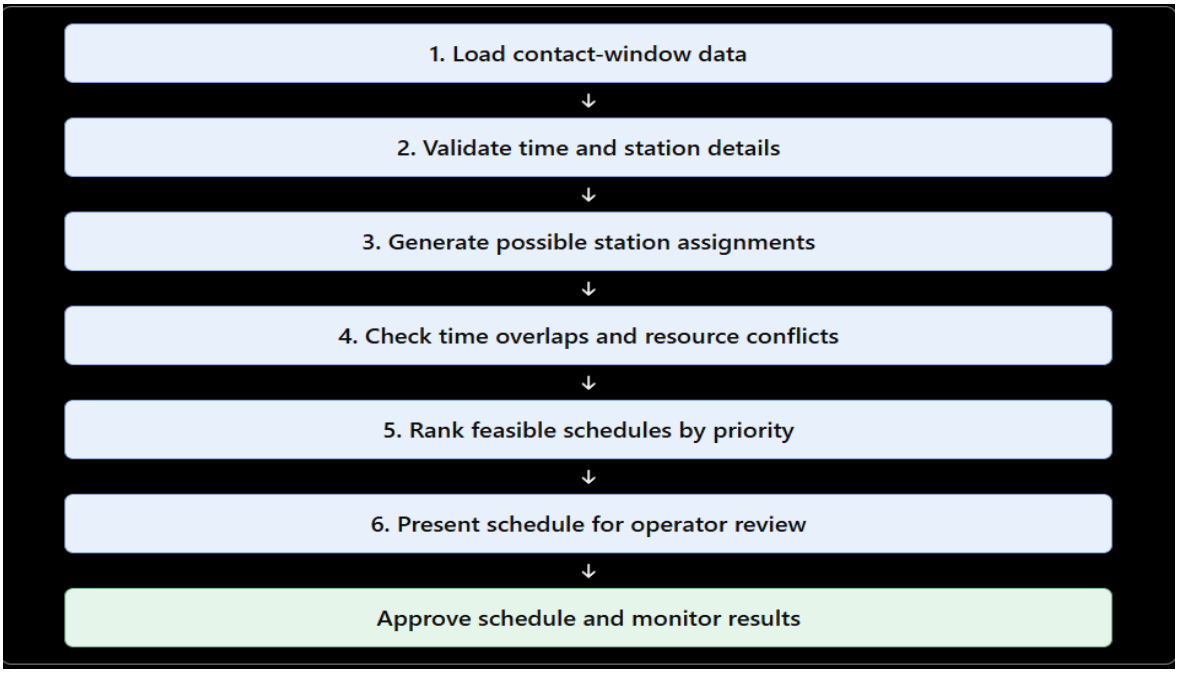


Figure 2: Satellite Contact Scheduling Workflow

1. Load contact-window data
2. Validate time and station details
3. Generate possible station assignments
4. Check time overlaps and resource conflicts
5. Rank feasible schedules by priority
6. Present schedule for operator review

Approve schedule and monitor results

Explanation

First, the system loads the available contact windows. It validates the start time, end time, station identifier, and other required information.

Next, it generates possible assignments and checks for overlapping time intervals at the same station. If conflicts exist, the system identifies them and suggests alternative assignments where possible.

The system then presents the proposed schedule to an authorized operator. The operator reviews and approves the schedule before it is treated as an approved plan.

5.3 Example of a Scheduling Conflict

Suppose Ground Station GS-01 has the following contact requests:

Contact	Start Time	End Time
Satellite A	10:00 AM	10:30 AM
Satellite B	10:20 AM	10:50 AM
Satellite C	11:00 AM	11:30 AM

Satellite A and Satellite B have overlapping contact windows from 10:20 AM to 10:30 AM. If the station cannot serve both requests simultaneously, the scheduling system must flag the conflict.

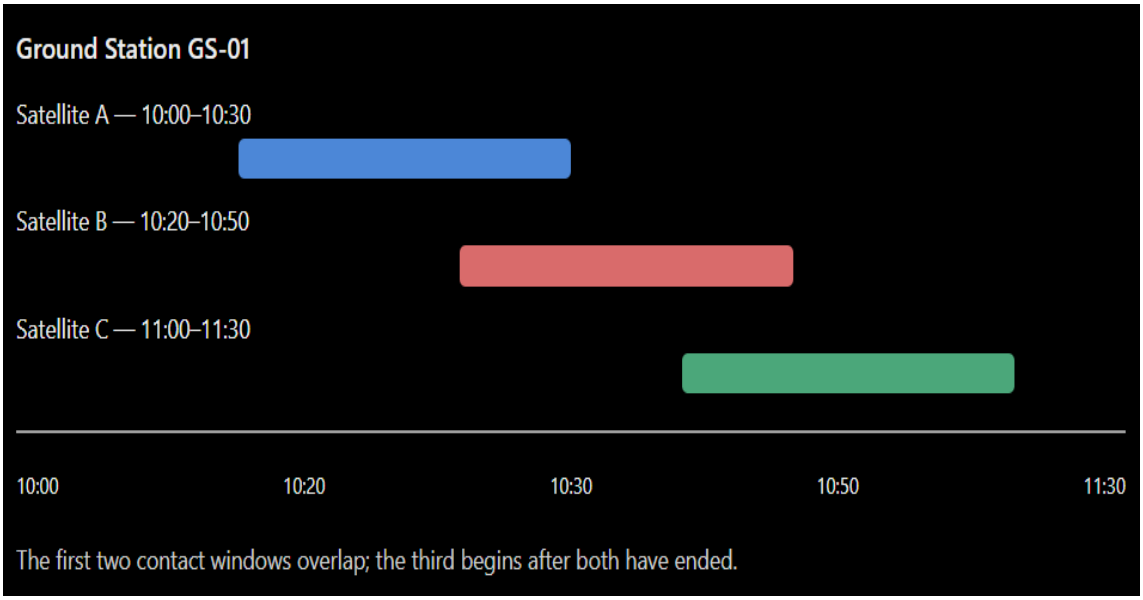


Figure 3: Illustrative Contact-Window Timeline

Ground Station GS-01

Satellite A — 10:00–10:30

Satellite B — 10:20–10:50

Satellite C — 11:00–11:30

10:00

10:20

10:30

10:50

11:30

The first two contact windows overlap; the third begins after both have ended.

The system can recommend moving Satellite B to another available station or an alternative valid time window. The final decision depends on station capability, mission priority, and operational constraints.

5.4 Health-Monitoring Workflow

The health-monitoring module checks station and communication-link readings against configured thresholds.

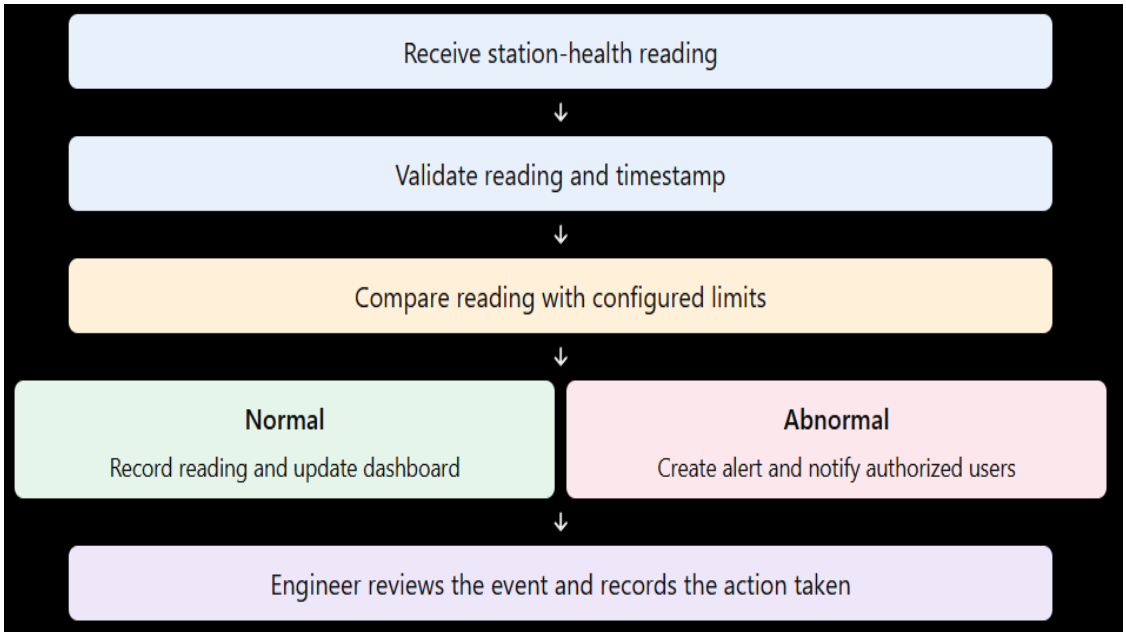


Figure 4: Station-Health Monitoring Flow

Receive station-health reading

Validate reading and timestamp

Compare reading with configured limits

Normal

Record reading and update dashboard

Abnormal

Create alert and notify authorized users

Engineer reviews the event and records the action taken

This process helps the team identify abnormal conditions quickly. Thresholds must be configured for the relevant measurements; the prototype does not assume that one threshold is suitable for every type of station.

5.5 Secure Data-Access Workflow

The system must protect operational information by checking the identity and permissions of each user.

Figure 5: Authentication and Authorization Flow

User submits login details

Authenticate user identity

Check assigned role and permissions

Permission denied

Reject request and record the event

Permission granted

Allow the authorized operation and record relevant activity

Authentication confirms who the user is. Authorization determines what the user is allowed to do. Both are necessary for secure system operation.

6. Epics and User Stories

6.1 Meaning of an Epic

An epic is a large functional requirement that can be divided into smaller user stories. Each user story describes a specific capability from the perspective of a user.

A common user-story format is:

As a user role, I want a capability, so that I can achieve a benefit.

The following epics describe the main requirements of the proposed system.

6.2 Epic 1: Ground-Station Management

Description: Manage ground-station details, capabilities, and availability.

User Story US-01: Register a ground station

As a system administrator, I want to register a ground station so that the system can maintain its details.

User Story US-02: View station availability

As a mission planner, I want to view the availability of each station so that I can select a suitable station for a satellite contact.

User Story US-03: Update station status

As a ground-station operator, I want to update the operational status of a station so that planners can avoid unavailable stations.

6.3 Epic 2: Satellite Scheduling

Description: Manage contact windows and detect scheduling conflicts.

User Story US-04: Add contact windows

As a mission planner, I want to add satellite contact windows so that the system can include them in the schedule.

User Story US-05: Detect scheduling conflicts

As a mission planner, I want the system to detect overlapping assignments so that conflicting requests can be resolved.

User Story US-06: Recommend alternative schedules

As a mission planner, I want to receive alternative feasible assignments so that I can reduce scheduling conflicts.

User Story US-07: Approve a schedule

As an authorized operator, I want to review and approve a proposed schedule so that only reviewed schedules become approved plans.

6.4 Epic 3: Telemetry Data Management

Description: Receive, validate, store, and retrieve telemetry information.

User Story US-08: Receive telemetry data

As a telemetry analyst, I want the system to receive telemetry records so that measurements can be monitored centrally.

User Story US-09: Validate telemetry records

As a telemetry analyst, I want invalid or incomplete records to be identified so that unreliable data can be investigated.

User Story US-10: View historical telemetry

As a telemetry analyst, I want to view previous readings so that I can compare current conditions with historical information.

6.5 Epic 4: Station-Health Monitoring

Description: Monitor operational indicators and identify abnormal readings.

User Story US-11: View station health

As a network engineer, I want to view the health status of each station so that I can identify stations requiring attention.

User Story US-12: Detect abnormal readings

As a network engineer, I want abnormal readings to be detected automatically so that potential problems can be investigated early.

6.6 Epic 5: Alerts and Notifications

Description: Notify authorized users about operational events.

User Story US-13: Generate an alert

As an operator, I want the system to generate an alert when a configured condition occurs so that I can respond promptly.

User Story US-14: View alert history

As an operator, I want to review previous alerts so that I can understand recurring operational problems.

6.7 Epic 6: Dashboard and Reports

Description: Present useful information in an understandable format.

User Story US-15: View the operations dashboard

As an operator, I want to see station status, upcoming contacts, and active alerts on one dashboard so that I can understand the current operational situation.

User Story US-16: Generate reports

As a project supervisor, I want to generate summary reports so that I can review system performance and operational activity.

6.8 Epic 7: Security and Access Control

Description: Protect the platform and restrict access according to user roles.

User Story US-17: Log in securely

As a registered user, I want to authenticate before accessing protected features so that unauthorized users cannot access the system.

User Story US-18: Enforce role permissions

As a system administrator, I want different roles to have appropriate permissions so that users can perform only authorized actions.

7. Acceptance Criteria for User Stories

7.1 Meaning of Acceptance Criteria

Acceptance criteria are the conditions that a user story must satisfy before it can be accepted as complete. They help developers understand what to implement and help testers verify whether the functionality meets the requirements.

The Given–When–Then format is commonly used to express acceptance criteria:

- Given: The initial situation or condition.
- When: The action performed by the user.
- Then: The expected result.

7.2 Acceptance Criteria for Selected User Stories

US-01: Register a Ground Station

- Given that an administrator is logged in,
- When the administrator enters valid station details and submits the form,
- Then the system must save the station and display a confirmation.

Additional conditions:

- Required fields must not be empty.
- Duplicate station identifiers must be rejected.
- Invalid input must produce a clear error message.

US-02: View Station Availability

- Given that station records exist,
- When a mission planner opens the station availability page,
- Then the system must display each station's current status and relevant availability information.

Additional conditions:

- Unavailable stations must be clearly identified.
- The displayed information must reflect the latest stored status.
- Empty results must be handled without crashing the application.

US-04: Add Satellite Contact Windows

- Given that a mission planner has the necessary permission,
- When the planner enters a valid contact window,
- Then the system must save the contact details.

Additional conditions:

- The end time must be later than the start time.
- Required satellite and station details must be provided.
- Invalid time values must be rejected.

US-05: Detect Scheduling Conflicts

- Given that two contact requests use the same station during overlapping periods,
- When the scheduling engine checks the requests,
- Then the system must identify and display the conflict.

Additional conditions:

- Non-overlapping requests must not be flagged as conflicts merely because they are on the same day.
- The conflict message must identify the affected station and contact requests.
- The system must not silently approve a conflicting schedule.

US-06: Recommend Alternative Schedules

- Given that a contact request conflicts with another assignment,
- When the scheduling engine evaluates available alternatives,
- Then it must display feasible alternatives that satisfy the configured constraints.

Additional conditions:

- Recommendations must respect station capability and availability.
- The system must explain when no feasible alternative is found.
- Recommendations must not be treated as approved schedules automatically.

US-08: Receive Telemetry Data

- Given that a telemetry record follows the expected data format,
- When the record is submitted,
- Then the system must validate and store the record.

Additional conditions:

- The timestamp and station identifier must be validated.
- Invalid records must be rejected or placed in a controlled error-handling process.
- The system must retain sufficient information to investigate failed ingestion.

US-12: Detect Abnormal Readings

- Given that a valid reading is received and monitoring thresholds are configured,
- When the reading exceeds an applicable threshold,
- Then the system must create an alert.

Additional conditions:

- Normal readings must not generate unnecessary abnormal-condition alerts.
- The alert must include the affected station, reading, and timestamp.
- Alert history must be available to authorized users.

US-15: View the Operations Dashboard

- Given that an authorized operator logs in,
- When the operator opens the dashboard,
- Then the system must display station status, upcoming contact windows, and relevant alerts.

Additional conditions:

- Information must be readable and logically organized.
- Empty sections must display appropriate messages.
- The dashboard must not expose information that the user is not authorized to access.

US-17: Log In Securely

- Given that a registered user has valid credentials,
- When the user submits those credentials,
- Then the system must authenticate the user and provide access permitted by the user's role.

Additional conditions:

- Invalid credentials must not grant access.
- Passwords must not be stored as plain text.
- Protected application functions must enforce authorization checks.

8. Product Backlog and Prioritization

8.1 Meaning of the Product Backlog

A product backlog is an ordered list of features, improvements, and requirements that may be implemented in the product. The product owner is responsible for ensuring that backlog items are clear and prioritized.

The backlog is not a fixed list. It can be updated when new requirements, technical difficulties, or stakeholder feedback arise.

8.2 MoSCoW Prioritization Technique

The MoSCoW technique divides requirements into four categories.

- Must Have: Essential for the initial working system.
- Should Have: Important, but the initial system can function without it.
- Could Have: Useful improvements that can be added if time permits.
- Won't Have for Now: Features deliberately excluded from the current release.

8.3 Prioritized Product Backlog

ID	Backlog Item	Priority	Reason
PB-01	User login and role permissions	Must Have	Protects access to the application
PB-02	Register and view ground stations	Must Have	Establishes station information
PB-03	Add and view contact windows	Must Have	Provides scheduling inputs
PB-04	Detect scheduling conflicts	Must Have	Addresses the main scheduling problem
PB-05	Receive and validate telemetry	Must Have	Enables basic telemetry monitoring
PB-06	Display station status dashboard	Must Have	Provides operational visibility
PB-07	Generate health alerts	Should Have	Improves awareness of abnormal conditions
PB-08	Recommend alternative schedules	Should Have	Helps resolve scheduling conflicts
PB-09	Search historical telemetry	Should Have	Supports investigation and comparison
PB-10	Generate downloadable reports	Could Have	Improves reporting convenience
PB-11	Advanced predictive maintenance	Could Have	Adds advanced analytical capability
PB-12	Direct integration with real satellite infrastructure	Won't Have for Now	Requires external infrastructure and authorization

The priorities above are proposed for the academic prototype. They can be revised after discussing the requirements with the project stakeholders.

8.4 Product Backlog Refinement

Product backlog refinement is the process of reviewing and improving backlog items before they are selected for a sprint.

During refinement, the team should:

1. Remove duplicate or unnecessary requirements.
2. Clarify ambiguous user stories.
3. Add acceptance criteria.
4. Identify dependencies between stories.
5. Estimate the effort required.
6. Reassess priorities.
7. Split stories that are too large to complete within a sprint.

For example, “Develop the complete scheduling system” is too broad for a single user story. It can be divided into adding contact windows, validating contact details, detecting overlaps, displaying conflicts, and approving schedules.

9. Sprint Planning and Sprint Backlog

9.1 Meaning of Sprint Planning

Sprint planning is a Scrum event in which the team decides why the sprint is valuable, what work can be completed, and how that work will be carried out.

The team considers:

- Product backlog priorities.
- Available team capacity.
- Technical dependencies.
- Acceptance criteria.
- The expected sprint goal.

For this project, a two-week sprint is assumed. The actual duration may be adjusted according to the course schedule

9.2 Sprint 1 Goal

Sprint Goal: Develop the basic ground-station management foundation and enable users to register stations, view station information, and inspect station availability.

Sprint 1 focuses on a small, demonstrable part of the system. Scheduling optimization and advanced telemetry analysis are left for later sprints.

9.3 Selected Sprint Backlog

Task ID	Task Description	Estimated Effort
T-01	Confirm station-management requirements	3 hours
T-02	Design the station data model	4 hours
T-03	Create the station registration interface	5 hours
T-04	Implement station registration logic	6 hours
T-05	Implement station listing and details	5 hours
T-06	Add station-status management	4 hours
T-07	Implement input validation	4 hours
T-08	Connect the interface to the database	5 hours
T-09	Test station-management functions	5 hours
T-10	Document and demonstrate the feature	3 hours
Total	10 tasks	44 hours

These figures are planning estimates, not measured development results. They assume the team has the necessary tools and a basic development environment.

9.4 Sprint Task Workflow

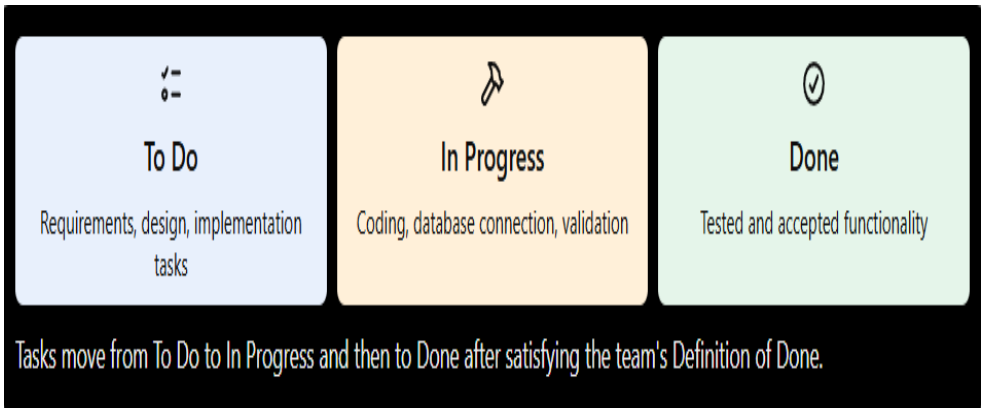


Figure 6: Sprint Task Progression

To Do

Requirements, design, implementation tasks

In Progress

Coding, database connection, validation

Done

Tested and accepted functionality

Tasks move from To Do to In Progress and then to Done after satisfying the team's Definition of Done.

9.5 Sprint Planning Activities

The team conducts sprint planning using the following steps:

1. The product owner explains the highest-priority backlog items.
2. The team discusses each story and its acceptance criteria.
3. The team estimates the effort and identifies dependencies.
4. Team members consider their availability during the sprint.
5. The team selects a realistic amount of work.
6. The team breaks selected stories into implementation and testing tasks.
7. The team agrees on the sprint goal.
8. The sprint backlog is recorded and monitored throughout the sprint.

The team should avoid selecting more work than it can reasonably complete. A sprint is successful when the agreed goal is achieved with acceptable quality, not simply when many tasks have been started.

10. Story-Point Estimation

10.1 Definition

Story points are a relative measure of the effort and complexity associated with completing a user story. They can reflect implementation effort, uncertainty, and technical complexity.

Story points are not the same as hours. For example, a story estimated at five points is generally considered more complex than a story estimated at two points, but this does not mean it will take exactly 2.5 times as long.

10.2 Fibonacci Estimation Scale

The Fibonacci sequence is commonly used in Agile estimation because uncertainty generally increases as the size of a task increases.

Story Points	Meaning	Example
1	Very small	Display a simple status label
2	Small	Add a basic input validation rule
3	Moderate	Create a station listing page
5	Medium to large	Implement station registration with database storage
8	Large	Develop basic scheduling conflict detection
13	Very large	Build a broad, complex scheduling subsystem

A story estimated at 13 points may be too large or uncertain for a sprint. The team should consider splitting it into smaller stories.

10.3 Illustrative Story-Point Estimates

User Story	Description	Story Points
US-01	Register a ground station	5
US-02	View station availability	3
US-03	Update station status	3
US-04	Add contact windows	5

US-05	Detect scheduling conflicts	8
US-08	Receive telemetry data	5
US-12	Detect abnormal readings	5
US-15	View the operations dashboard	5
US-17	Implement secure login	5

These values are initial estimates for planning. The development team should review them together and revise them when it learns more about the requirements.

10.4 Planning Poker

Planning Poker is a collaborative estimation technique.

The procedure is:

1. The product owner explains the user story.
2. Team members discuss the requirements and acceptance criteria.
3. Each team member privately selects a story-point estimate.
4. All estimates are revealed.
5. If estimates differ, team members explain their reasoning.
6. The team discusses the major sources of complexity.
7. Members estimate again until a reasonable team estimate is agreed upon.

Planning Poker reduces the influence of one person's opinion and encourages the team to identify hidden implementation difficulties.

10.5 Velocity

Velocity is the number of story points completed by a team during a sprint, based on the team's established definition of completed work.

For example, if a team completes stories worth 15 points in Sprint 1, its observed velocity for that sprint is 15 points.

Velocity can help with future planning after several sprints. It should not be used to compare unrelated teams or treated as a measure of individual productivity.

11. Definition of Ready and Definition of Done

11.1 Definition of Ready

The Definition of Ready describes when a user story is sufficiently clear to be considered for a sprint.

A user story is ready when:

- Its objective is understandable.
- Its user role and expected benefit are identified.
- Acceptance criteria are documented.
- Dependencies are known or sufficiently understood.
- The team has enough information to estimate the work.
- The story is appropriately sized for the sprint.

- Required sample data or test information is available.

For example, the story “Detect scheduling conflicts” is not fully ready if the team has not defined what constitutes an overlap or whether a station can handle multiple simultaneous contacts.

11.2 Definition of Done

The Definition of Done describes the quality conditions that a completed story must satisfy.

A user story is considered done when:

- The agreed functionality is implemented.
- The code follows the project's coding standards.
- Input validation and error handling are implemented.
- Relevant tests have passed.
- Security and authorization checks are applied where required.
- The feature is integrated with the rest of the application.
- The documentation is updated where necessary.
- Acceptance criteria are satisfied.
- The completed feature can be demonstrated.

A feature should not be marked as done merely because its code has been written. Testing, integration, and acceptance are also necessary.

12. Future Sprint Roadmap

The project can be developed over six illustrative sprints. The roadmap may change depending on feedback, technical complexity, and available development time.

Sprint	Main Goal	Expected Deliverable
Sprint 1	Establish station management	Register, view, and update stations
Sprint 2	Implement contact-window management	Add and view satellite contact requests
Sprint 3	Develop scheduling conflict detection	Identify overlapping station assignments
Sprint 4	Implement telemetry monitoring	Receive, validate, and view telemetry
Sprint 5	Add alerts and security improvements	Health alerts and role-based access
Sprint 6	Complete dashboard, testing, and reporting	Integrated prototype and final demonstration

12.1 Sprint 2: Contact-Window Management

The team develops forms and database operations for contact requests. It validates time ranges, station references, and required fields.

Expected outcome: Users can add and view valid contact windows.

12.2 Sprint 3: Scheduling Conflict Detection

The team implements time-overlap checks and presents conflicting requests clearly. It also begins supporting alternative assignments where feasible.

Expected outcome: The system can demonstrate conflict detection using sample data.

12.3 Sprint 4: Telemetry Monitoring

The team implements telemetry ingestion, validation, storage, and basic historical viewing.

Expected outcome: Users can inspect valid telemetry readings and identify rejected records.

12.4 Sprint 5: Alerts and Security

The team introduces configurable health thresholds, alert history, authentication, and role-based permissions.

Expected outcome: Authorized users can review alerts, while protected operations are restricted appropriately.

12.5 Sprint 6: Integration and Final Testing

The team integrates the modules, resolves defects, improves dashboard usability, and prepares the demonstration.

Expected outcome: A tested academic prototype with documentation and evidence of completed acceptance criteria.

13. Testing Strategy

Software testing verifies that the application behaves as expected and satisfies its requirements.

Testing should be carried out throughout development rather than postponed until the final sprint.

13.1 Unit Testing

Unit testing verifies individual functions or components.

Examples:

- Test whether a station identifier is validated correctly.
- Test whether an invalid time range is rejected.
- Test whether two overlapping time intervals are identified.
- Test whether a telemetry reading is compared with the correct threshold.

13.2 Integration Testing

Integration testing checks whether different components work correctly together.

Examples:

- Verify that station registration saves information in the database.
- Verify that the scheduling module reads contact windows correctly.
- Verify that abnormal telemetry readings create alerts.
- Verify that the dashboard displays the information returned by the backend.

13.3 System Testing

System testing evaluates the complete application as a whole.

Examples:

- Register stations, add contact windows, and generate a schedule.
- Submit telemetry data and verify that the dashboard updates.
- Generate an alert and check whether it appears in alert history.
- Log in with different roles and verify access permissions.

13.4 Acceptance Testing

Acceptance testing checks whether the developed features meet the agreed acceptance criteria.

For example, the scheduling module should identify two overlapping requests that use the same station when simultaneous service is not supported. A non-overlapping request should not be reported as a conflict merely because it occurs on the same date.

13.5 Sample Test Cases

Test ID	Test Scenario	Expected Result
TC-01	Register a station with valid details	Station is saved successfully
TC-02	Register a station with missing required fields	Validation error is displayed
TC-03	Enter an end time before the start time	Contact request is rejected
TC-04	Add two overlapping contacts for one station	Conflict is identified
TC-05	Add two non-overlapping contacts for one station	No overlap conflict is reported
TC-06	Submit an invalid telemetry record	Record is rejected or handled by the defined error process
TC-07	Submit a reading above its configured threshold	An alert is generated
TC-08	Log in with invalid credentials	Access is denied
TC-09	Attempt an unauthorized operation	The operation is blocked

TC-10	Open the dashboard with valid permissions	Authorized information is displayed

14. Project Risks and Mitigation

Risk management helps the team identify problems that may affect the project's quality, schedule, security, or feasibility.

Risk	Possible Impact	Mitigation
Incomplete requirements	Rework and confusion	Clarify user stories and acceptance criteria early
Incorrect scheduling logic	Invalid assignments	Test overlapping and non-overlapping time intervals
Invalid telemetry records	Incorrect monitoring results	Validate formats, timestamps, and required fields
Database integration issues	Data loss or application errors	Test database operations early
Excessive sprint workload	Incomplete deliverables	Select realistic work based on team capacity
Unauthorized data access	Security exposure	Apply authentication and server-side authorization
Delayed stakeholder feedback	Incorrect feature priorities	Conduct regular sprint reviews
Limited development time	Reduced feature coverage	Prioritize essential features and defer advanced features
Inadequate testing	Defects in demonstrations	Reserve time for unit, integration, and acceptance testing
Overly complex design	Difficult implementation and maintenance	Build a small working prototype before adding advanced features

14.1 Technical Risk Management

The scheduling engine is a major technical component because an incorrect result could suggest that two incompatible contact requests can use the same station simultaneously.

To reduce this risk, the team should test boundary conditions, overlapping intervals, station capabilities, and unavailable stations. Scheduling recommendations should be reviewed before approval.

Telemetry monitoring also requires careful validation. Missing, delayed, or invalid data should not automatically be interpreted as a confirmed station failure. The system should distinguish data-quality problems from actual abnormal measurements.

14.2 Project Management Risk Management

The team should track sprint progress, discuss blockers during the Daily Scrum, and revise the plan when necessary.

If a story proves larger than expected, the team should consider splitting it or moving non-essential work to a future sprint. This is preferable to delivering unfinished functionality without testing.

15. Expected Results and Success Metrics

The success of the proposed system should be assessed using measurable results rather than general claims.

Area	Success Measure
Station management	Valid station records can be registered, stored, and retrieved
Scheduling	Configured overlapping requests are correctly identified
Telemetry	Valid records are accepted and invalid records are handled safely
Health monitoring	Readings that meet configured alert conditions generate alerts
Dashboard	Relevant station, schedule, and alert information is displayed
Security	Unauthorized requests are rejected
Testing	Agreed acceptance tests pass
Documentation	User stories, diagrams, test cases, and implementation notes are available

The team should establish actual results through testing. Numerical improvements in scheduling efficiency, failure-detection time, or operational reliability should not be claimed until supported by measured evidence.

16. Conclusion

The Distributed Satellite Ground Station Telemetry and Scheduling Network provides a structured approach to coordinating satellite communication activities across multiple ground stations. The proposed system combines station management, satellite contact scheduling, telemetry validation, health monitoring, notifications, dashboards, and access control within one platform.

The use of Agile Scrum makes the project easier to plan, develop, and demonstrate incrementally. Epics divide the overall system into functional areas, while user stories express requirements from the perspective of different stakeholders. Acceptance criteria establish measurable conditions for feature completion, and product backlog prioritization ensures that essential functionality receives attention first.

Sprint planning converts prioritized requirements into achievable development tasks. Story-point estimation supports discussions about complexity, while the Definition of Ready and Definition of Done help maintain clarity and quality. Unit testing, integration testing, system testing, and acceptance testing provide evidence that the implemented features behave as expected.

The initial prototype can be developed using simulated ground-station, contact-window, and telemetry data. This makes it feasible for an academic Software Engineering project without requiring direct access to operational satellite infrastructure. Advanced scheduling, predictive monitoring, and real-world integrations can be considered after the basic modules have been implemented and tested.

In conclusion, the project demonstrates how Agile principles can be applied to a complex, multi-module software system. Through incremental development, clear requirements, regular feedback, and systematic testing, the team can deliver a practical prototype that demonstrates improved coordination, visibility, and management of distributed satellite ground-station operations.

17. References

1. Schwaber, K., and Sutherland, J. The Scrum Guide. Scrum.org.
2. Beck, K., et al. Manifesto for Agile Software Development. Agile Manifesto.
3. Pressman, R. S., and Maxim, B. R. Software Engineering: A Practitioner's Approach. McGraw-Hill.
4. Sommerville, I. Software Engineering. Pearson.
5. IEEE. ISO/IEC/IEEE 29148: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering.
6. IEEE. ISO/IEC/IEEE 29119 series: Software Testing.

Note: These references provide general background for Agile, requirements engineering, and software testing. The project-specific diagrams, user stories, sprint estimates, and proposed workflows in this assignment are illustrative designs for the academic prototype.